D3 Silent errors

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D3 Silent errors 3
D3-L1 Join row explosion and nulls in filters 3
D3-L2 Type coercion and timezone drift 7
D3-L3 Mutable defaults and the truthiness of zero 12
Track D best-practice checklist 17

Appendix: solutions and answer keys 18

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D3 Silent errors

Roadmap: Every month. Join explosion, nulls in filters, type coercion, timezone drift, mutable defaults.

D3-L1 Join row explosion and nulls in filters

Concept

Some bugs raise no error. The job succeeds and the numbers are wrong. Two of the most common come from joins and from nulls.

Join row explosion. A left join of respondents to a region lookup should keep one row per respondent. If the lookup holds a key twice, say `N` for "North" and "North (old)", every respondent in `N` appears twice. Weighted totals double for that region. Nothing fails.

Check the grain around every join:

- Count rows before and after. A many-to-one join must not change the count.
- In pandas, pass `validate="many_to_one"`. A duplicate key then raises `MergeError: Merge keys are not unique in right dataset`. Checked on pandas 3.0.2.
- In Spark, count duplicate keys in the lookup before joining: `dim.groupBy(key).count().filter("count > 1")`.

Nulls behave differently in pandas and Spark. Checked on pandas 3.0.2 and PySpark 4.0.4:

Operation	pandas	Spark and SQL
Filter `region != "N"`	Keeps rows where region is missing	**Drops** rows where region is null
Join on a null key	**Matches** null to null	Never matches a null key
`x NOT IN (subquery)` where the subquery holds a null	Not applicable	Returns **no rows**

In Spark and SQL, a comparison with null gives null, not True or False, and a filter keeps only True. So `F.col("region_code") != "N"` silently drops every respondent with no region. Say what you mean:

- Keep nulls: `(F.col("region_code") != "N") | F.col("region_code").isNull()`, or `~F.col("region_code").eqNullSafe("N")`.
- Exclude a list that might contain nulls: use `NOT EXISTS` or a `left_anti` join, not `NOT IN`.

In pandas, the surprise runs the other way: null keys match each other. Drop or fill null keys before a merge unless you want that.

Why it matters

A regional reach table showed the North at 118% weighted reach. The region lookup had gained a second row for code `N` when a boundary changed. The left join doubled every northern respondent. A row count check before and after the join, or `validate="many_to_one"`, would have stopped the job at the join.

Worked example: weak code

```
1  def add_region(respondents: pd.DataFrame, regions: pd.DataFrame) ->
     pd.DataFrame:
2      """Attach region names."""
1      merged = respondents.merge(regions, on="region_code", how="left")
2      return merged[merged["region"] != "Unknown"]
```

1. **Line 3.** No grain check. A duplicate code doubles rows, and null codes match a null row in regions.
2. **Line 4.** A filter that also changes the row count, mixed into the join step.

Worked example: strong code

```
1  def add_region(respondents: pd.DataFrame, regions: pd.DataFrame) ->
     pd.DataFrame:
2      """Attach region names. One output row per respondent.
3
4      Raises:
5          pandas.errors.MergeError: If a region code appears twice in
     regions.
6      """
1      lookup = regions.dropna(subset=["region_code"])
2      merged = respondents.merge(lookup, on="region_code", how="left",
     validate="many_to_one")
3      if len(merged) != len(respondents):
10         raise ValueError(f"join changed rows: {len(respondents)} ->
     {len(merged)}")
```

```
    return merged
```

1. **Line 7.** pandas matches null keys to each other. Drop them from the lookup.
2. **Line 8.** pandas raises MergeError if the lookup has a duplicate key.
3. **Line 9.** An explicit grain check. It also works where validate is not available.

Check row counts across every join. Know how your engine treats nulls in filters and keys, and write the null case explicitly.

Recap card

- **Check for join row explosion** Row counts before and after. pandas: validate='many_to_one'. Spark: count duplicate keys in the lookup first.
- **col != 'x' with nulls: pandas vs Spark** pandas keeps missing values. Spark and SQL drop nulls, because the comparison is null.
- **Null join keys: pandas vs Spark** pandas merge matches null to null. Spark and SQL never match null keys.
- **NOT IN with a null in the list** Returns no rows. Use NOT EXISTS or a left_anti join.

Practice

Exercise 1. Predict the output (level 1)

What does this print? (pandas 3)

```python
import pandas as pd

resp = pd.DataFrame({"id": [1, 2, 3], "code": ["N", "S", "N"]})
regions = pd.DataFrame({"code": ["N", "N", "S"], "name": ["North", "North (old)", "South"]})
print(len(resp.merge(regions, on="code", how="left")))
```

Exercise 2. Multiple choice (level 2)

In PySpark, `respondents.filter(F.col("region_code") != "N")` runs on 1,000 rows: 300 are `N`, 50 are null. How many rows remain?

- a) 650
- b) 700
- c) 350
- d) 1,000

Why?

- r1) null != 'N' evaluates to null, and a filter keeps only rows where the condition is true.
- r2) Spark drops nulls from every DataFrame before filtering.

Exercise 3. Multiple choice (level 2)

`SELECT * FROM respondents WHERE region_code NOT IN (SELECT code FROM excluded_regions)` returns no rows. `excluded_regions` holds `'N'` and one null. Why?

- a) x NOT IN (..., NULL) is never true, because x <> NULL is null
- b) NOT IN is not supported in Spark SQL
- c) The subquery returns too many rows
- d) Every respondent is in region N

Why?

- r1) NOT IN expands to x <> 'N' AND x <> NULL. The second term is null, so the whole condition is never true.
- r2) Subqueries ignore nulls.

Exercise 4. Write the function (level 3)

Write `safe_left_join(facts: pd.DataFrame, lookup: pd.DataFrame, key: str) -> pd.DataFrame`. Drop rows with a null key from `lookup` first. If a key appears more than once in the remaining lookup, raise `ValueError` naming the duplicated keys in sorted order. Return the left join, which must have exactly `len(facts)` rows. Null keys in `facts` must get missing lookup values, not match anything.

Your code must pass these checks:

- Raise ValueError listing duplicated keys in sorted order, like ['N', 'W'].
- pandas matches null keys to each other. Drop null keys from the lookup so a null fact key gets no name.
- Return one row per fact row, in the original order.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Left join that keeps the grain of facts.
def safe_left_join(
    facts: pd.DataFrame,
    lookup: pd.DataFrame,
    key: str,
) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


```

```
def safe_left_join(facts, lookup, key):
    return facts.merge(lookup, on=key, how="left")
```

Exercise 5. Spot the bug (level 3)

This step should remove test respondents (`panel == "test"`) and keep everyone else, including respondents with no panel recorded. 4% of real respondents vanish. Tap the line, then fix it.

```
from pyspark.sql import functions as F

live = respondents.filter(F.col("panel") != "test")
live.write.mode("overwrite").saveAsTable("research_silver.nat_live")
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas 3.0.2: validate='many_to_one' raises MergeError 'Merge keys are not unique in right dataset; not a many-to-one merge'; != keeps missing values; merge matches null keys. (verified: Local run on pandas 3.0.2 (Pyodide pin), September 2026.)
- PySpark 4.0.4: != filter drops nulls; eqNullSafe keeps them when negated; null join keys do not match; NOT IN with a null in the subquery returns no rows; NOT EXISTS and left_anti do not. (verified: Local run on PySpark 4.0.4, September 2026.)

D3-L2 Type coercion and timezone drift

Concept

Two more silent errors: values that change type, and times that change meaning with the zone.

Type coercion. Checked on pandas 3.0.2, PySpark 4.0.4 and Python 3.13:

- `pd.read_csv` turns `respondent_id` `0123` into the integer `123`. The leading zero is gone, and joins to the panel file fail. Pass `dtype={"respondent_id": "string"}` for identifiers.
- An integer column with one missing value becomes `float64` in pandas. `1` becomes `1.0`, and string keys built from it read `"1.0"`.
- Strings sort as text: `sorted(["10", "9", "2"])` gives `['10', '2', '9']`. Wave "W10" sorts before "W2". Pad your ids, or sort on a number.

- Spark's `df.union(other)` matches columns **by position**, not by name. A delivery with `wave_id` and `respondent_id` swapped unions without error, with the values in the wrong columns. Use `unionByName`.
- Under ANSI mode, Spark casts a string column compared with a number: `F.col("age") > 9` raises `CAST_INVALID_INPUT` on the value `'x'`. Cast explicitly with `try_cast` first.

Timezone drift. A timestamp without a zone is a guess.

- `datetime.utcnow()` returns a **naive** datetime and is deprecated from Python 3.12. Comparing naive with aware raises `TypeError: can't compare offset-naive and offset-aware datetimes`. Use `datetime.now(timezone.utc)`.
- Spark's `TIMESTAMP` type is an instant, displayed in the **session time zone** (`spark.sql.session.timeZone`). `F.to_date` on `2026-06-30T23:30:00Z` gives 30 June in UTC and **1 July** in Europe/London. A wave cut-off at midnight moves respondents between waves.
- The same text without an offset, `'2026-06-30 23:30:00'`, parses to instants 4 hours apart in UTC and America/New_York sessions.
- Some local times do not exist. pandas refuses to localise `2026-03-29 01:30` to Europe/London: the clocks skipped that hour.

Store instants in UTC. Convert to local time only for display, or for a business rule that names its zone, such as "fieldwork closes at midnight UK time".

Why it matters

A wave closed at midnight UK time on 30 June. The pipeline ran with the session time zone set to UTC and assigned every interview completed between 23:00 and 00:00 UTC on 30 June to wave W14. In British Summer Time those interviews happened after midnight, in W15. 340 respondents moved wave. Both waves' reach figures shifted by up to 1.5 points.

Worked example: weak code

```
from datetime import datetime

def in_wave(completed_at: datetime, close_at: datetime) -> bool:
    """Was the interview completed before the wave closed?"""
    return completed_at < close_at

close = datetime(2026, 6, 30, 23, 59, 59)
in_wave(datetime.utcnow(), close)
```

1. **Line 7.** Naive: is this UK time or UTC? The code cannot tell, and nor can a reader.

2. **Line 8.** Deprecated since Python 3.12, and naive too.

Worked example: strong code

```
1  from datetime import datetime, timedelta, timezone
2
3  BST = timezone(timedelta(hours=1), "BST")
4
5  def in_wave(completed_at: datetime, close_at: datetime) -> bool:
6      """Was the interview completed before the wave closed?
7
8      Raises:
9          ValueError: If either datetime is naive.
10     """
11     if completed_at.tzinfo is None or close_at.tzinfo is None:
12         raise ValueError("datetimes must carry a time zone")
13     return completed_at < close_at
14
15 close = datetime(2026, 7, 1, 0, 0, tzinfo=BST)
16 in_wave(datetime.now(timezone.utc), close)
```

1. **Line 3.** The business rule names its zone. In production, use zoneinfo.ZoneInfo("Europe/London") to handle the clock change.
2. **Line 11.** Refuse naive values instead of guessing.
3. **Line 16.** Aware UTC now. Aware datetimes compare correctly across zones.

Read identifiers as strings, union by name, and keep every timestamp aware and stored in UTC.

Recap card

- **pd.read_csv and leading zeros** Default parsing makes 0123 into 123. Pass dtype='string' for identifiers.
- **Spark union vs unionByName** union matches columns by position, silently. unionByName matches by name.
- **datetime.utcnow()** Deprecated from Python 3.12 and naive. Use datetime.now(timezone.utc).
- **Spark TIMESTAMP and the session time zone** An instant, shown and converted in spark.sql.session.timeZone. to_date can move a row to another day.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
1 waves = ["W10", "W9", "W2"]
2 print(sorted(waves))
```

Exercise 2. Multiple choice (level 2)

A Spark session has `spark.sql.session.timeZone` set to `Europe/London`. An interview's `completed_at` is the instant `2026-06-30T23:30:00Z`. What does `F.to_date("completed_at")` return?

- a) 2026-07-01
- b) 2026-06-30
- c) It depends on the cluster's operating system clock
- d) NULL

Why?

- r1) TIMESTAMP is an instant. to_date uses the session time zone, where 23:30 UTC is 00:30 on 1 July in BST.
- r2) Spark always converts to local time before storing.

Exercise 3. Fill the gap (level 2)

Replace the deprecated call with an aware UTC timestamp.

```
from datetime import datetime, timezone

run_timestamp = datetime._____(1)(_____(2))
```

Exercise 4. Write the function (level 3)

Write `load_delivery(csv_text: str) -> pd.DataFrame`. Read the CSV with `respondent_id` and `postcode_area` as strings, keeping leading zeros. Convert `final_weight` to float. If any non-empty weight fails to parse, raise `ValueError` naming how many. Empty weights stay missing.

What you need to know.

`pd.to_numeric(series, errors="coerce")` converts text to numbers. A value it cannot parse becomes missing instead of raising an error:

```
s = pd.Series(["1.5", "abc", None], dtype="string")
pd.to_numeric(s, errors="coerce")   # 1.5, <NA>, <NA>
```

A value that was present before and missing after failed to parse. Compare `s.notna()` with the parsed result's `.isna()` to find those rows.

Your code must pass these checks:

- Keep identifiers as strings so leading zeros survive.
- Weights are floats. An empty weight stays missing.
- Two weights failed to parse. Raise ValueError with the count, instead of turning them into missing values.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
ID_COLUMNS = ...

# Read a delivery CSV with identifiers as strings and weights as floats.
def load_delivery(csv_text: str) -> pd.DataFrame: ...
```

Starter code:

```
import io

import pandas as pd


def load_delivery(csv_text):
    return pd.read_csv(io.StringIO(csv_text))
```

Exercise 5. Spot the bug (level 3)

A redelivery arrives with its columns in a different order. After this step, some respondent ids read `W14`. Tap the line, then fix it.

```
from pyspark.sql import functions as F

combined = original.union(redelivery)
combined = combined.withColumn("loaded_at", F.current_timestamp())
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas 3.0.2: read_csv drops leading zeros by default; int with None becomes float64; tz_localize rejects a nonexistent DST time. (verified: Local run on pandas 3.0.2, September 2026.)
- Python 3.13: utcnow() raises a DeprecationWarning and is naive; comparing naive and aware datetimes raises TypeError. (verified: Local run on CPython 3.13.12, September 2026.)
- PySpark 4.0.4: union matches by position; to_date depends on session time zone; naive strings parse relative to the session zone; ANSI casts a string compared with an int and raises CAST_INVALID_INPUT on 'x'. (verified: Local run on PySpark 4.0.4, September 2026.)

D3-L3 Mutable defaults and the truthiness of zero

Concept

Two Python habits produce wrong results with no error message. Both appear in code written by people and by AI assistants, including your reference pipeline.

Mutable default arguments. Python evaluates a default value once, when it defines the function. Every call that relies on the default shares that one object. Write `def add_check(result, results=[])`, and each call appends to the same list. The second validation run returns the first run's checks as well.

The fix is a `None` sentinel:

```
def add_check(result: str, results: list[str] | None = None) -> list[str]:
    results = [] if results is None else results
    results.append(result)
    return results
```

ruff flags the pattern as rule B006. Treat that warning as a bug.

Truthiness of numbers. `if value:` is `False` for `None`, and also for `0`, `0.0`, empty strings and empty containers. When zero is a valid measurement, such as a 0% reach, a zero weight or zero rows, a truthiness test treats it as missing.

Your reference `check_wave_stability` does this:

```
if current_pct and prior_pct:
```

A metric that falls from 12% to 0% skips the comparison. The largest possible drop produces no warning.

Test for the case you mean. For missing, write `is None`. For zero, write `== 0`.

```
if current_pct is not None and prior_pct is not None:
```

The same rule covers type hints. `error_message: str = None` claims the value is always a string. Write `str | None = None`, and mypy then makes you handle the `None` case.

Why it matters

A questionnaire routing error removes one question for a whole wave. Its weighted percentage drops from 12% to 0.0%. The wave-over-wave stability check runs `if current_pct and prior_pct:`, treats 0.0 as missing and skips the metric. QA reports PASS. The dashboard publishes 0% reach for that question, and a client asks why the channel "lost its whole audience" in a week.

The check existed to catch exactly this change. One truthiness test disabled it for the one value that mattered most.

Worked example: weak code

```
2  def compare_waves(current: dict, prior: dict, tolerance: float = 0.10,
     flags=[]):
2      """Flag metrics whose share moved more than tolerance between
     waves."""
3      for metric, current_pct in current.items():
4          prior_pct = prior.get(metric)
3          if current_pct and prior_pct:
6              if abs(current_pct - prior_pct) > tolerance:
7                  flags.append(metric)
8      return flags
```

1. **Line 1.** Bare `dict` hides the value type. The values are floats or None.
2. **Line 1.** The default list is created once. Flags from one call leak into the next.
3. **Line 5.** A share of 0.0 is falsy, so the biggest possible drop is never compared.

Worked example: strong code

```
1  def compare_waves(
1      current: dict[str, float | None],
3      prior: dict[str, float | None],
4      tolerance: float = 0.10,
5  ) -> list[str]:
6      """Flag metrics whose share moved more than tolerance between waves.
7
8      A metric is skipped only when a share is missing. A share of 0.0 is
     compared.
9      """
2      flags: list[str] = []
11     for metric, current_pct in current.items():
12         prior_pct = prior.get(metric)
3          if current_pct is None or prior_pct is None:
14             continue
15         if abs(current_pct - prior_pct) > tolerance:
16             flags.append(metric)
17     return flags
```

1. **Line 2.** The hint says a share can be missing, so mypy checks every use.
2. **Line 10.** A new list on every call. No state leaks between runs.
3. **Line 13.** Skips only missing values. 0.0 goes on to the comparison.

Both fixes are one line. Both bugs pass every test that uses only non-zero values and a single call.

Recap card

- **When does Python evaluate a default argument?** Once, when it defines the function. A list or dict default is shared by every call.
- **Safe pattern for an optional list argument** Default to None. Inside: results = [] if results is None else results.
- **What is wrong with `if current_pct and prior_pct:`?** 0.0 is falsy, so a real share of zero is treated as missing. Use `is not None`.
- **Type hint for an optional string that defaults to None** str | None = None. Never str = None.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
def add_flag(flag, flags=[]):
    flags.append(flag)
    return flags

add_flag("null_weights")
print(add_flag("row_count"))
```

Exercise 2. Multiple choice (level 2)

`current_pct = 0.0` and `prior_pct = 0.12`. The tolerance is 0.10. What does `check_wave_stability` from the reference code do with this metric?

- a) Flags it as WARN, because the change is 0.12.
- b) Marks it PASS.
- c) Leaves it out of the results.
- d) Raises a ZeroDivisionError.

Why?

- r1) `if current_pct and prior_pct:` is False when current_pct is 0.0, so the block never runs.
- r2) The function filters out metrics below 1%.

Exercise 3. Fill the gap (level 3)

Fix both signatures. The error message is optional, and the flags list must not be shared between calls.

```
def log_run(status: str, error_message: _____(1) = None) -> None: ...

```

```
3 def collect(flag: str, flags: list[str] | None = _____(2)) ->
    list[str]: ...
```

Exercise 4. Spot the bug (level 3)

This check should warn when a share moves by more than the tolerance. A drop from 12% to 0% produces no warning. Tap the line with the bug, then fix the function.

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct and prior_pct:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 4)

Write `compare_waves(current, prior, tolerance=0.10)`. Both arguments map metric names to a share or `None`. Return the metrics whose share changed by more than `tolerance`, in the order of `current`. Skip a metric only when either share is `None` or the metric is missing from `prior`. Calling the function twice must not carry results over.

Your code must pass these checks:

- A drop from 0.12 to 0.0 must be flagged. Test for None, not for truth.
- A None share must be skipped, and an unchanged share must not be flagged.
- A metric missing from prior must be skipped, not raise KeyError. Use `prior.get(metric)`.
- The second call returned flags from the first call. Create a new list inside the function.
- Return flags in the order of `current`, without sorting.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Flag metrics whose share moved more than tolerance between waves.
2 def compare_waves(
3     current: dict[str, float | None],
4     prior: dict[str, float | None],
5     tolerance: float = 0.1,
6 ) -> list[str]: ...
```

Starter code:

```python
def compare_waves(
    current: dict[str, float | None],
    prior: dict[str, float | None],
    tolerance: float = 0.10,
) -> list[str]:
    ...
```

Exercise 6. Refactor (level 5)

A validator collects check results with the helper below. The second contract validation in a job reports the first contract's failures too. Refactor `add_result` so each validation starts empty, keep the signature compatible for callers that pass a list, and add type hints and a docstring.

Your code must pass these checks:

- Two calls without a list must not share results.
- When the caller passes a list, append to that list and return it.
- An empty list from the caller was replaced. `results or []` treats an empty list as missing. Test `results is None`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Append one check result and return the list.
def add_result(
    name: str,
    passed: bool,
    results: list[dict[str, object]] | None = None,
) -> list[dict[str, object]]: ...
```

Starter code:

```python
def add_result(name, passed, results=[]):
    results.append({"check": name, "passed": passed})
    return results
```

Sources

- ruff rule B006 flags mutable default arguments. (verified: ruff 0.16.9 run in this repo reports B006 on `def f(x=[])`.)

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D3-L1 Join row explosion and nulls in filters

D3-L1 exercise 1

Hints. (1) How many rows in regions match code N? (2) How many respondents have code N? (3) 2 x 2 + 1.

Answer.

```
5
```

Each N respondent matches two lookup rows. 4 rows for N, 1 for S. The left join multiplied the grain.

Common wrong answers.

- `3`: A left join keeps every left row, but each one appears once per matching right row. Code N matches twice.
- `4`: Two respondents have code N, and each matches two rows: 2 x 2 + 1 = 5.

D3-L1 exercise 2

Hints. (1) What is null != 'N' in SQL logic? (2) Null, which is not true. (3) 1,000 - 300 - 50.

Answer. a) 650 Because: r1) null != 'N' evaluates to null, and a filter keeps only rows where the condition is true.

Three-valued logic: comparisons with null give null. Add `| F.col("region_code").isNull()` to keep them.

Common wrong answers.

- `b`: That is pandas behaviour. In Spark the 50 null rows are dropped too.
- `c`: That keeps N and null. The filter removes N.
- `d`: The filter removes at least the 300 N rows.
- `a|r2`: Right answer, wrong reason. Spark keeps nulls in general. Only the comparison yields null here.

D3-L1 exercise 3

Hints. (1) Expand NOT IN into ANDs. (2) What is x <> NULL? (3) Use NOT EXISTS or a left_anti join.

Answer. a) x NOT IN (..., NULL) is never true, because x <> NULL is null Because: r1) NOT IN expands to x <> 'N' AND x <> NULL. The second term is null, so the whole condition is never true.

Use `NOT EXISTS` or a `left_anti` join. Both treat a null in the exclusion list as "no match".

Common wrong answers.

- `b`: It runs. It just returns no rows when the list contains a null.
- `c`: Two rows is fine. The null is the problem.
- `d`: Even an S respondent fails the null comparison.
- `a|r2`: Right answer, wrong reason. The subquery keeps the null, and that is what breaks NOT IN.

D3-L1 exercise 4

Hints. (1) `lookup.dropna(subset=[key])` removes null keys. (2) `value_counts()` then `counts[counts > 1]` finds duplicates. (3) Check `len(joined) == len(facts)` before returning.

Answer.

```python
import pandas as pd


def safe_left_join(facts: pd.DataFrame, lookup: pd.DataFrame, key: str) -> pd.DataFrame:
    """Left join that keeps the grain of facts.

    Args:
        facts: Rows to keep, one per output row.
        lookup: Attributes to attach, at most one row per key.
        key: Join column.

    Returns:
        The joined frame with len(facts) rows.

    Raises:
        ValueError: If lookup holds a key more than once, or the grain changes.
    """
    clean = lookup.dropna(subset=[key])
    counts = clean[key].value_counts()
    duplicated = sorted(counts[counts > 1].index.tolist())
    if duplicated:
```

```
22          raise ValueError(f"duplicate keys in lookup: {duplicated}")
23      joined = facts.merge(clean, on=key, how="left")
24      if len(joined) != len(facts):
25          raise ValueError(f"join changed rows: {len(facts)} ->
    {len(joined)}")
26      return joined
```

Report duplicates, never drop them silently: which "North" is right is a business decision. Null keys leave the lookup before the merge.

D3-L1 exercise 5

Hints. (1) What is null != 'test' in Spark? (2) Null, so the row is dropped. (3) eqNullSafe treats null as a comparable value.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 live = respondents.filter(~F.col("panel").eqNullSafe("test"))
4 live.write.mode("overwrite").saveAsTable("research_silver.nat_live")
```

`~F.col("panel").eqNullSafe("test")` is true for nulls, so respondents with no panel stay.

Common wrong answers.

- Line 4: The write stores what the filter kept. The rows were lost one line earlier.

D3-L2 Type coercion and timezone drift

D3-L2 exercise 1

Hints. (1) Compare 'W10' and 'W2' character by character. (2) '1' < '2'. (3) W10 first.

Answer.

```
1 ['W10', 'W2', 'W9']
```

Text sorts by character. Pad ids (`W02`, `W09`, `W10`) or sort by the number: `sorted(waves, key=lambda w: int(w[1:]))`.

Common wrong answers.

- `['W2', 'W9', 'W10']`: That is numeric order. Strings compare character by character, and '1' comes before '2'.
- `['W9', 'W2', 'W10']`: Strings sort ascending by character. 'W1...' comes first.

D3-L2 exercise 2

Hints. (1) What is 23:30 UTC in London in summer? (2) BST is UTC+1. (3) 00:30 the next day.

Answer. a) 2026-07-01 Because: r1) TIMESTAMP is an instant. to_date uses the session time zone, where 23:30 UTC is 00:30 on 1 July in BST.

Date boundaries depend on the zone. Set the session time zone deliberately, or convert with a named zone in the rule itself.

Common wrong answers.

- `b`: That is the date in UTC. The session time zone here is London, one hour ahead in summer.
- `c`: Spark uses the session time zone setting, not the OS clock.
- `d`: The conversion succeeds. It just gives a different day.
- `a|r2`: Right answer, wrong reason. Spark stores the instant. It applies the zone when converting or displaying.

D3-L2 exercise 3

Hints. (1) Which method takes a time zone argument? (2) Pass the UTC zone object. (3) `now` and `timezone.utc`.

Answer.

```
from datetime import datetime, timezone

run_timestamp = datetime.now(timezone.utc)
```

`datetime.now(timezone.utc)` returns an aware datetime in UTC.

Common wrong answers.

- g1 = `utcnow`: utcnow() is deprecated from Python 3.12 and returns a naive datetime.

D3-L2 exercise 4

Hints. (1) Pass `dtype=` to read_csv for the id columns. (2) Read the weight as a string first, then `pd.to_numeric(..., errors="coerce")`. (3) Failed = present before, missing after.

Answer.

```
import io

import pandas as pd

ID_COLUMNS = {"respondent_id": "string", "postcode_area": "string"}

```

```python

def load_delivery(csv_text: str) -> pd.DataFrame:
    """Read a delivery CSV with identifiers as strings and weights as floats.

    Args:
        csv_text: CSV text with respondent_id, postcode_area and final_weight.

    Returns:
        The delivery as a DataFrame.

    Raises:
        ValueError: If any non-empty weight cannot be parsed.
    """
    df = pd.read_csv(io.StringIO(csv_text), dtype={**ID_COLUMNS, "final_weight": "string"})
    raw = df["final_weight"]
    parsed = pd.to_numeric(raw, errors="coerce")
    failed = int((raw.notna() & parsed.isna()).sum())
    if failed:
        raise ValueError(f"{failed} weights could not be parsed")
    return df.assign(final_weight=parsed.astype("float64"))
```

Read as text, convert explicitly, and count what the conversion lost. Silent coercion hides bad data.

D3-L2 exercise 5

Hints. (1) How does union match columns? (2) By position. (3) unionByName matches by name.

Answer. The bug is on line 3.

Fixed version:

```python
from pyspark.sql import functions as F

combined = original.unionByName(redelivery)
combined = combined.withColumn("loaded_at", F.current_timestamp())
```

`union` pairs the first column with the first column, whatever their names. `unionByName` pairs by name, and fails if a name is missing.

Common wrong answers.

- Line 4: Adding a timestamp cannot move ids into the wave column. The union already mixed them.

D3-L3 Mutable defaults and the truthiness of zero

D3-L3 exercise 1

Hints. (1) When is the default `[]` created? (2) Both calls use the default. Is it the same list? (3) The list is created once, when Python defines the function.

Answer.

```
['null_weights', 'row_count']
```

The default list is created when Python defines the function. Both calls append to that one list, so the second call returns both flags.

Common wrong answers.

- `['row_count']`: That is what the author intended. The default list is created once, so the second call appends to the list the first call used.
- `['row_count', 'null_weights']`: `append` adds to the end. The first call's flag is already in the list.

D3-L3 exercise 2

Hints. (1) Evaluate `0.0 and 0.12` in your head. (2) `0.0` is falsy, like `None`. (3) The whole block, including `results.append`, is skipped.

Answer. c) Leaves it out of the results. Because: r1) `if current_pct and prior_pct:` is False when current_pct is 0.0, so the block never runs.

`0.0 and 0.12` evaluates to `0.0`, which is falsy. The block that compares and appends never runs, so the metric vanishes from the results and no warning appears.

Common wrong answers.

- `a`: That is what the check should do. Look at the `if` line before the comparison: 0.0 is falsy.
- `b`: Nothing is appended for this metric, so it cannot be marked PASS. The `if` skips it.
- `d`: The check subtracts, it does not divide. No error appears, which is the problem.
- `c|r2`: Right outcome, wrong reason. There is no 1% filter. The truthiness test treats 0.0 as missing.

D3-L3 exercise 3

Hints. (1) The first default is None, so the type must allow None. (2) The second default must be immutable. (3) `str | None` for the first gap and `None` for the second.

Answer.

```
1 def log_run(status: str, error_message: str | None = None) -> None: ...
2
3 def collect(flag: str, flags: list[str] | None = None) -> list[str]: ...
```

`error_message: str | None = None` states that the value may be missing. `flags: list[str] | None = None` avoids a shared list. Create the list inside the function when it is `None`.

Common wrong answers.

- g1 = `str: str = None` claims the value is always a string. mypy in strict mode rejects it. Write `str | None`.
- g2 = `[]`: A list default is created once and shared by every call. Use `None` and create the list inside the function.

D3-L3 exercise 4

Hints. (1) Which values make `if current_pct and prior_pct:` False? (2) 0.0 is one of them. (3) Replace the truthiness test with `is not None` on both values.

Answer. The bug is on line 3.

Fixed version:

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
     tolerance: float = 0.10) -> str:
2     """Return WARN, PASS, or SKIP when a share is missing."""
3     if current_pct is not None and prior_pct is not None:
4         change = abs(current_pct - prior_pct)
5         return "WARN" if change > tolerance else "PASS"
6     return "SKIP"
```

Line 3 treats 0.0 as missing. Use `if current_pct is not None and prior_pct is not None:` so that only missing shares skip the comparison.

Common wrong answers.

- Line 4: The subtraction is correct. The code never reaches it when a share is 0.0.
- Line 5: The comparison with the tolerance is correct. Look at what stops the code reaching it.

D3-L3 exercise 5

Hints. (1) Start with `flags: list[str] = []` inside the function. (2) Read the prior share with `prior.get(metric)`, which gives None for a missing metric. (3) Skip with `if current_pct is None or prior_pct is None: continue`, then compare with the tolerance.

Answer.

```python
def compare_waves(
    current: dict[str, float | None],
    prior: dict[str, float | None],
    tolerance: float = 0.10,
) -> list[str]:
    """Flag metrics whose share moved more than tolerance between waves.

    A metric is skipped only when a share is missing. A share of 0.0 is compared.
    """
    flags: list[str] = []
    for metric, current_pct in current.items():
        prior_pct = prior.get(metric)
        if current_pct is None or prior_pct is None:
            continue
        if abs(current_pct - prior_pct) > tolerance:
            flags.append(metric)
    return flags
```

Create the list inside the function. Use `prior.get` so a new metric gives `None`. Skip only on `None`, then compare the absolute change with the tolerance.

D3-L3 exercise 6

Hints. (1) Default `results` to None. (2) Create a new list only when `results is None`. (3) `results or []` looks similar but replaces an empty list the caller passed. Use `is None`.

Answer.

```python
def add_result(
    name: str, passed: bool, results: list[dict[str, object]] | None = None
) -> list[dict[str, object]]:
    """Append one check result and return the list.

    Args:
        name: Check name.
        passed: Whether the check passed.
        results: List to append to. A new list is created when omitted.

    Returns:
        The list with the new result.
    """
```

```
    results = [] if results is None else results
    results.append({"check": name, "passed": passed})
    return results
```

Default to `None`, create the list when `results is None`, then append and return. Avoid `results or []`: an empty list the caller passed is falsy, so the function would append to a new list and the caller would never see the result.